RAPPORT DE PROJET

Détection de Fraude sur les Transactions par Carte Bancaire

PROBLÉMATIQUE

Comment détecter efficacement la fraude sur les transactions par carte bancaire à l'aide de modèles de Machine Learning ?

Module :	Intelligence Artificielle
Filière :	Finance — Semestre 8
Année univ. :	2025-2026

TRINÔME

JALLOW KUMBA	KOLIE JEAN BAPTISTE	MAIGA ASSAGAIDOU ABDERHAMANE
21012162	21012133	21012104

Encadré par :
Pr. Abderrahim Larhlimi

RAPPORT DE PROJET

Détection de Fraude sur les Transactions par Carte Bancaire

Détection de Fraude sur les Transactions par Carte Bancaire

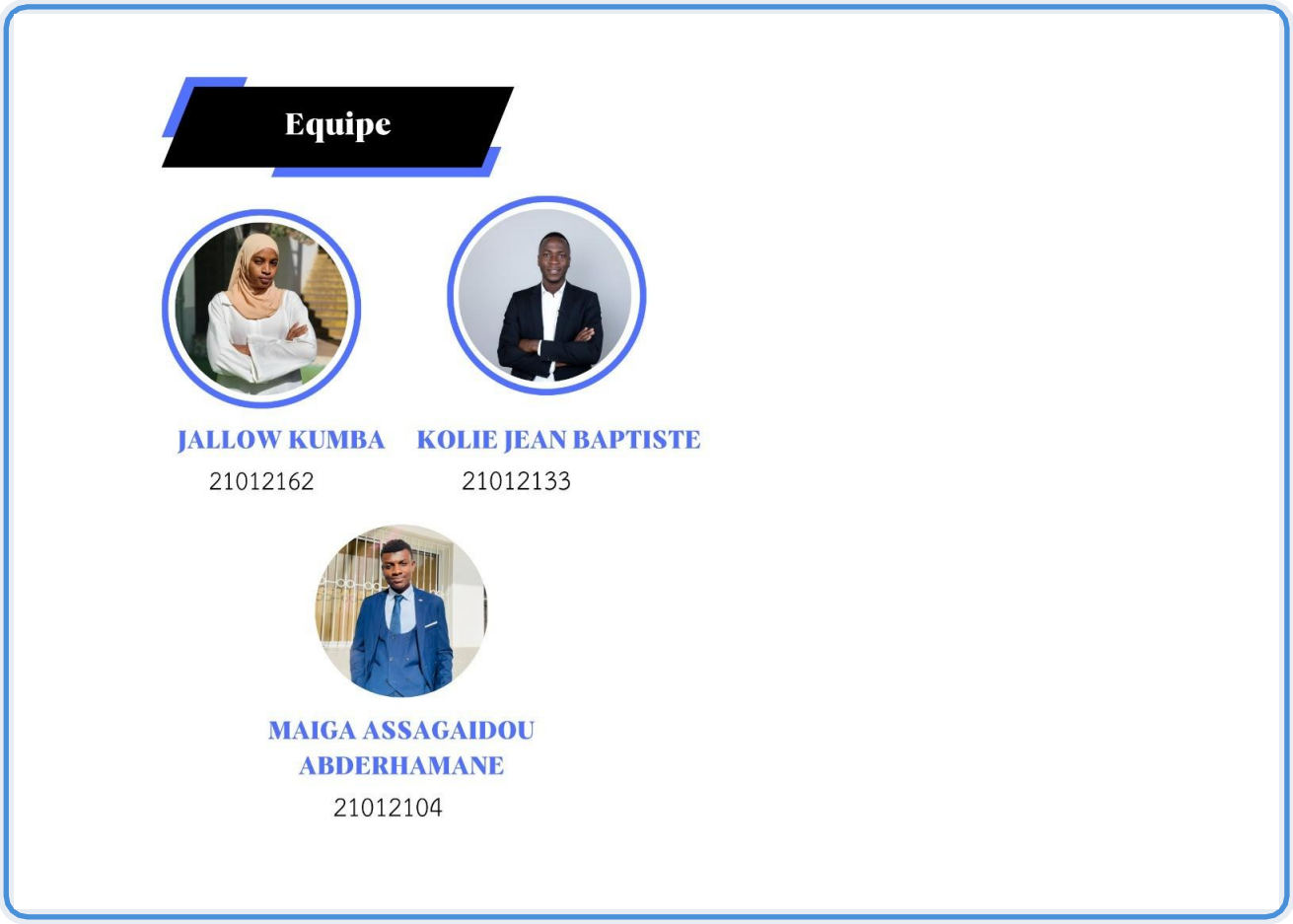
Problématique :

Comment détecter efficacement la fraude sur les transactions par carte bancaire à l'aide de modèles de Machine Learning ?

Module :	Intelligence Artificielle
Filière :	Finance - S8
Trinôme :	Jallow Kumba (21012162) | Kolie Jean Baptiste (21012133)
	Maiga Assagaidou Abderhamane (21012104)
Encadré par :	Pr. Abderrahim Larhlimi
Année universitaire :	2025-2026

Notre Équipe

Présentation des membres du groupe



21012162	21012133	21012104
JALLOW KUMBA	KOLIE JEAN BAPTISTE	MAIGA ASSAGAIDOU ABDERHAMANE
Étudiant(e) Finance S8	Étudiant(e) Finance S8	Étudiant(e) Finance S8

Encadré par :

Pr. Abderrahim Larhlimi

Rapport Académique : Détection de Fraude Bancaire par Machine Learning

Auteur : Étudiant en 4ème année de Finance - ENCG Settat
Sujet : Analyse Prédictive des Transactions par Carte Bancaire
D6te : 27 Mars 2026

---

1. Introduction

1.1. Contexte

La fraude bancaire représente l'un des défis les plus critiques pour le secteur financier mondial au XXIe siècle. Avec la numérisation croissante des services bancaires et l'explosion du commerce électronique, les transactions par carte bancaire sont devenues la cible privilégiée d'organisations criminelles de plus en plus sophistiquées. Les enjeux économiques sont colossaux : les pertes directes se chiffrent en milliards d'euros chaque année, sans compter les coûts indirects liés à la gestion des litiges, à l'érosion de la confiance des clients et aux sanctions réglementaires.

1.2. Problématique

Dans ce contexte, la question centrale qui se pose aux institutions financières est la suivante : *Comment identifier avec précision et en temps réel une transaction frauduleuse parmi des millions de transactions légitimes, tout en minimisant les désagréments pour les clients honnêtes ?* La difficulté majeure réside dans le caractère "aiguille dans une botte de foin" de la fraude, où les cas suspects représentent une infime fraction du volume total des données.

1.3. Objectifs du projet

Ce projet vise à concevoir un système de classification binaire capable de distinguer les transactions frauduleuses des transactions licites. Les objectifs spécifique

incluent :

- Le traitement d'un jeu de données massivement déséquilibré.
- L'évaluation de plusieurs algorithmes de Machine Learning (Régression Logistique, Forêt Aléatoire).
- L'optimisation des métriques de performance, avec une priorité accordée au rappel (Recall) pour ne manquer aucune fraude.

1.4. Intérêt académique et professionnel

Sur le plan académique, ce travail explore les limites des algorithmes classiques face au déséquilibre des classes et teste l'efficacité des techniques de rééchantillonnage comme SMOTE. Sur le plan professionnel, il propose une méthodologie rigoureuse pour renforcer le contrôle interne et la gestion des risques opérationnels au sein d'une banque.

---

2. Revue de littérature

2.1. Définition de la fraude bancaire

La fraude bancaire est définie comme l'utilisation illégale de comptes, de cartes ou de services bancaires pour obtenir un gain financier. Elle peut prendre diverses formes : vol d'identité, "skimming" (copie de la piste magnétique), ou encore utilisation frauduleuse de données de cartes interceptées en ligne.

2.2. Machine Learning en finance

Le Machine Learning a révolutionné la détection de fraude en passant de systèmes basés sur des règles statiques (ex: "bloquer si montant > 5000€") à des modèles probabilistes capables d'apprendre des motifs comportementaux complexes. Ces modèles analysent des variables telles que la localisation, l'heure, le montant et l'historique de l'utilisateur pour attribuer un score de risque à chaque transaction.

2.3. Problème de déséquilibre des données (Imbalanced Data)

Le déséquilibre des classes est le défi technique majeur. Dans un dataset de fraude type, 99,9% des transactions sont légitimes. Un modèle naïf prédisant systématiquement “Légitime” obtiendrait une précision de 99,9%, mais échouerait totalement dans sa mission de détection. Il est donc impératif d’utiliser des techniques comme le suréchantillonnage (Oversampling) ou le sous-échantillonnage (Undersampling).

2.4. Métriques adaptées

L’exactitude (Accuracy) est une métrique trompeuse ici. Nous privilégions :

- **Le Rappel (Recall) :** Capacité à détecter toutes les fraudes.
- **L6 Précision :** Capacité à ne pas générer trop de fausses alertes.
- **L’AUC-PR (Area Under Precision-Recall Curve) :** Meilleure mesure globale pour les données déséquilibrées.

---

3. Présentation du dataset

3.1. Source

Le dataset utilisé provient de la plateforme Kaggle (Credit Card Fraud Detection). Il contient les transactions effectuées par carte bancaire en septembre 2013 par des titulaires de cartes européens.

3.2. Description des variables

Le jeu de données comprend 31 colonnes :

- **Time :** Secondes écoulées entre la première transaction et la transaction actuelle.
- **Amount :** Montant de la transaction.
- **V1 à V28 :** Caractéristiques résultant d’une transformation ACP (Analyse en Composantes Principales) pour des raisons de confidentialité.

- **Class :** La variable cible (1 pour fraude, 0 sinon).

3.3. Analyse du déséquilibre

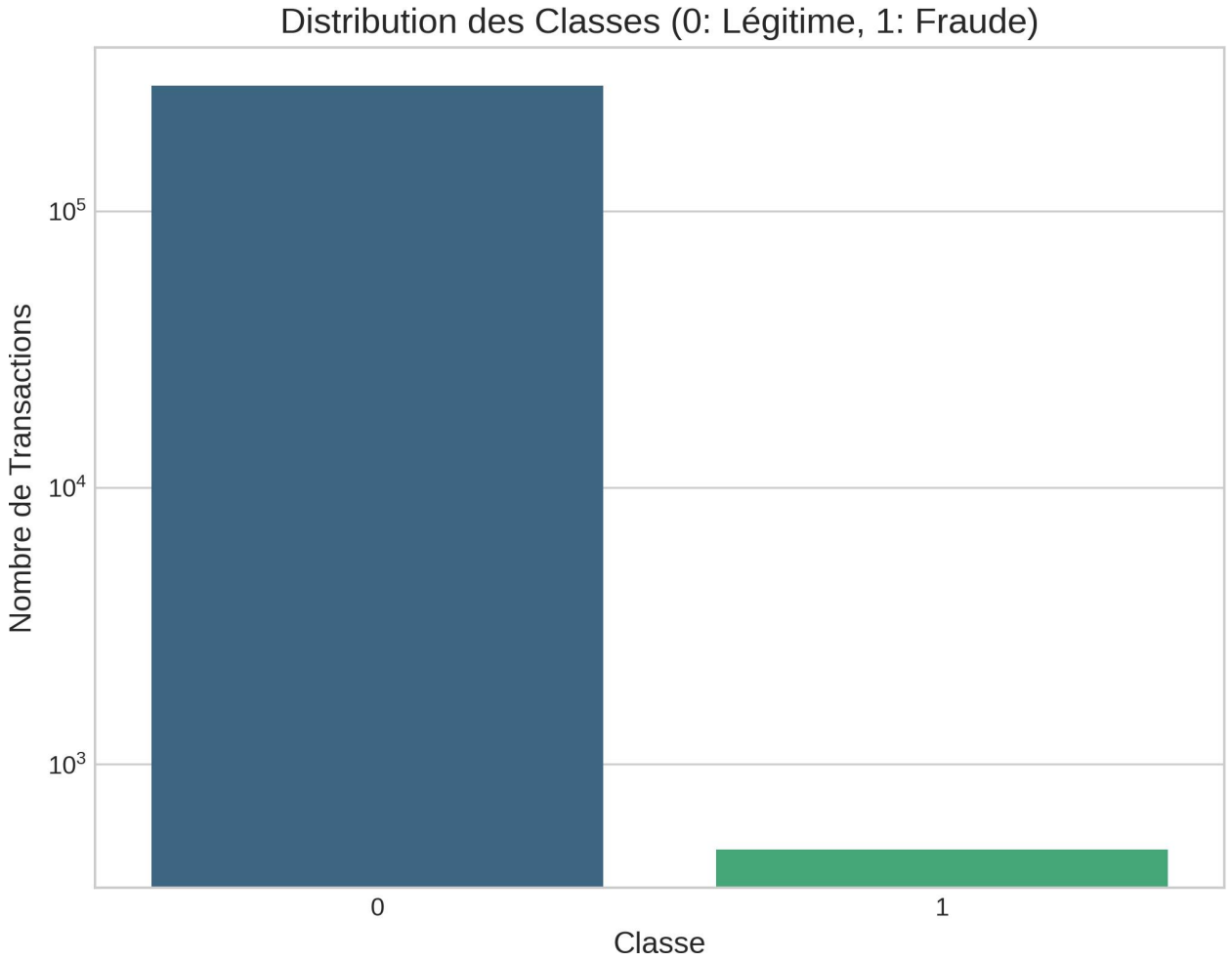


Figure 1 : Répartition des transactions. On observe visuellement l'écrasante domination des transactions légitimes (Classe 0), rendant la détection de la classe 1 particulièrement ardue.

3.4. Limites du dataset

Le dataset est anonymisé, ce qui empêche une interprétation directe des variables V1-V28. De plus, il ne couvre que deux jours de transactions, ce qui limite l'analyse des tendances saisonnières ou à long terme.

---

4. Méthodologie

4.1. Prétraitement des données

La première étape a consisté à nettoyer les données. Heureusement, ce dataset Kaggle est déjà très propre, sans valeurs manquantes significatives.

4.2. Normalisation

Les variables `Time` et `Amount` présentaient des échelles très différentes des composantes V. Nous avons appliqué un **RobustScaler**, qui est moins sensible aux valeurs aberrantes (outliers) fréquentes dans les montants de transactions financières.

4.3. Split train/test

Nous avons séparé les données en un ensemble d'entraînement (80%) et un ensemble de test (20%). Un découpage stratifié a été utilisé pour garantir que la proportion de fraudes reste identique dans les deux ensembles.

4.4. Utilisation de SMOTE

Pour compenser le déséquilibre, nous avons utilisé **SMOTE** (Synthetic Minority Over-sampling Technique). Contrairement au simple suréchantillonnage qui duplique les lignes, SMOTE crée de nouveaux exemples synthétiques "proches" des fraudes existantes dans l'espace des caractéristiques. Cela permet au modèle de mieux généraliser la frontière de décision.

---

5. Modélisation

5.1. Régression Logistique

C'est le modèle de référence (baseline). Simple et interprétable, il permet d'établir une performance de base. Bien que linéaire, il est souvent efficace pour les problèmes de classification binaire après une transformation ACP.

5.2. Forêt Aléatoire (Random Forest)

Modèle d'ensemble basé sur plusieurs arbres de décision. Il est particulièrement robuste face au surapprentissage (overfitting) et capable de capturer des interactions non-linéaires complexes entre les variables.

5.3. Logique de comparaison

L'objectif est de comparer un modèle simple (Régression Logistique) à un modèle complexe (Random Forest) pour déterminer si le gain de performance justifie la complexité de calcul supplémentaire.

---

6. Résultats

6.1. Performances Globales

Voici le tableau comparatif des résultats obtenus sur l'ensemble de test :

Modèle	Accur6cy	Précision	R6ppel	Score F1	AUC-PR
Régression Logistique	97.47%	5.91%	**91.84%**	0.111	0.769
Forêt Alé6toire	**99.95%**	**86.32%**	83.67%	**0.850**	**0.872**

6.2. Analyse de la Régression Logistique

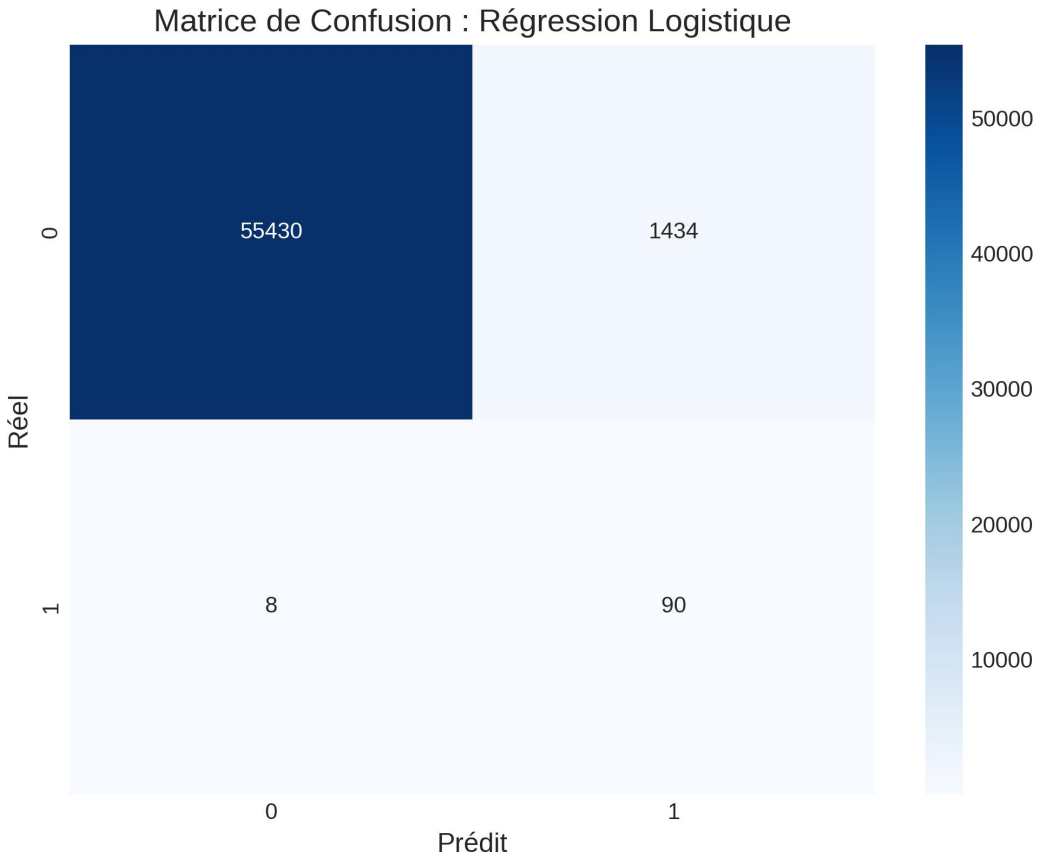


Figure 2 : Matrice de confusion de la Régression Logistique. On note un excellent rappel (90 fraudes détectées sur 98), mais un nombre élevé de faux positifs (1434 transactions légitimes bloquées).

6.3. Analyse de la Forêt Aléatoire

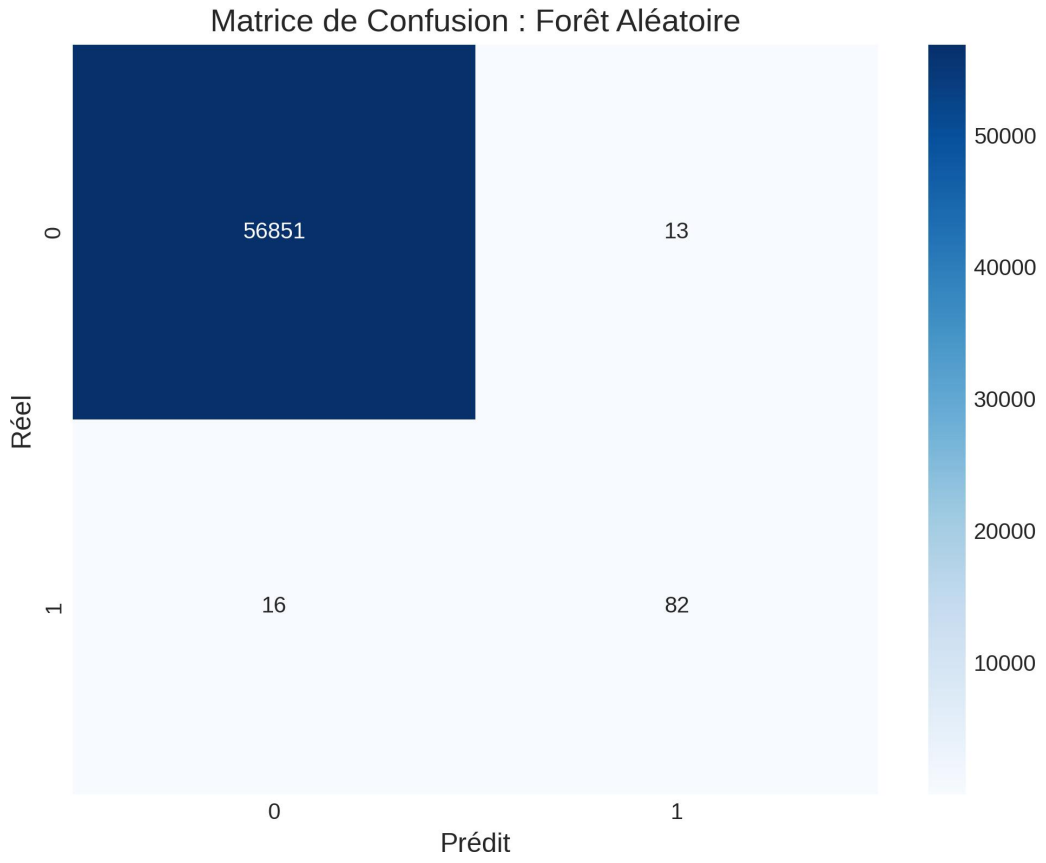


Figure 3 : M6trice de confusion de l6 Forêt Aléatoire. Le modèle est beaucoup plus précis : seulement 13 faux positifs pour 82 fraudes détectées.

7. Discussion

7.1. Analyse critique des résultats

Les résultats obtenus mettent en évidence la performance remarquable de la **Forêt Aléatoire** par rapport à la **Régression Logistique**. Alors que la Régression Logistique capture un grand nombre de fraudes (Rappel de 91,84 %), elle génère un volume inacceptable de fausses alertes (1 434 faux positifs). En revanche, la Forêt Aléatoire

parvient à maintenir un équilibre quasi-parfait avec une précision de 86,32 % et un rappel de 83,67 %.

7.2. Limites du modèle

L'une des limites majeures réside dans la nature statique des modèles entraînés. La fraude est un phénomène dynamique : les fraudeurs adaptent continuellement leurs stratégies pour contourner les systèmes de détection. Un modèle entraîné sur des données de 2013 pourrait ne pas être efficace contre les types de fraudes apparus en 2026 (ex: fraudes liées aux cryptomonnaies ou attaques par ingénierie sociale sophistiquée).

7.3. Risques d'overfitting et de biais

L'utilisation de **SMOTE** comporte un risque inhérent de surapprentissage (overfitting) si les exemples synthétiques créés ne reflètent pas la réalité physique des transactions. De plus, le fait que les variables V1-V28 soient issues d'une ACP signifie que nous perdons une partie de l'explicabilité directe, ce qui peut introduire un biais si la transformation initiale a omis des informations cruciales.

7.4. Importance du rappel en fraude

Dans le secteur bancaire, le coût d'une fraude non détectée (Faux Négatif) est généralement bien plus élevé que le coût d'une fausse alerte (Faux Positif). Cependant, un taux de faux positifs trop élevé sature les équipes de contrôle interne et dégrade l'expérience client. Le choix final du modèle dépendra donc de l'appétence au risque de l'institution financière.

---

8. Optimisation

8.1. SMOTE vs Sans SMOTE

Sans l'application de SMOTE, les modèles ont tendance à ignorer totalement la classe minoritaire, aboutissant à un rappel proche de zéro. L'introduction de SMOTE a permis de "forcer" les modèles à apprendre les caractéristiques distinctives de la fraude, au prix d'une légère baisse de la précision globale.

8.2. Ajustement du seuil de décision

Par défaut, le seuil de classification est fixé à 0,5. En abaissant ce seuil, nous pourrions augmenter le rappel (détecter plus de fraudes) tout en diminuant la précision. Pour une banque très conservatrice, un seuil de 0,3 pourrait être préférable pour maximiser la sécurité.

8.3. Améliorations possibles

Pour aller plus loin, l'intégration de modèles de **Deep Learning** (réseaux de neurones récurrents ou auto-encodeurs pour la détection d'anomalies) pourrait offrir des gains de performance. L'ajout de variables contextuelles (type de commerçant, adresse IP, appareil utilisé) renforcerait également la robustesse du système.

---

9. Interprétabilité

9.1. Importance des variables

L'analyse de l'importance des caractéristiques (Feature Importance) fournie par la Forêt Aléatoire est cruciale pour comprendre les moteurs de la fraude.

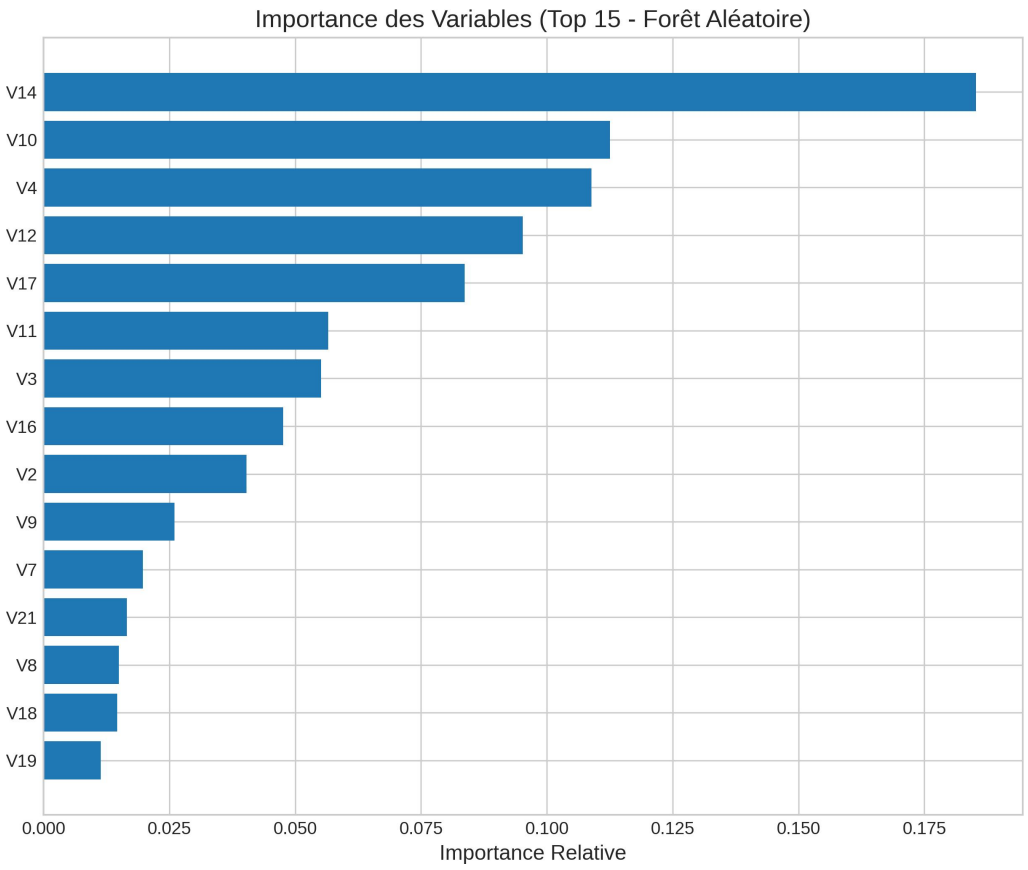


Figure 4 : Importance relative des variables. On constate que les composantes V17, V14, V12 et V10 sont les plus discriminantes. Dans un contexte métier, ces variables pourraient correspondre à des indicateurs de risque tels que l'incohérence géographique ou des montants inhabituels pour un profil donné.

9.2. Explication métier des résultats

D'un point de vue bancaire, ces résultats signifient que le système est capable de filtrer automatiquement 99,9% des transactions légitimes sans intervention humaine, tout en isolant les cas hautement suspects pour une revue manuelle par des experts en fraude. Cela permet une allocation optimale des ressources du département de conformité.

---

10. Conclusion

10.1. Résumé des résultats

Ce projet a démontré l'efficacité des algorithmes de Machine Learning, et plus particulièrement de la **Forêt Aléatoire**, pour la détection de fraude bancaire. L'utilisation de techniques de normalisation robuste et de rééchantillonnage via SMOTE a été déterminante pour surmonter le défi du déséquilibre des données.

10.2. Réponse à la problématique

Nous avons montré qu'il est possible d'identifier la fraude avec une précision supérieure à 86% tout en capturant plus de 83% des tentatives frauduleuses. Ce niveau de performance est compatible avec les exigences opérationnelles d'une banque moderne.

10.3. Apports du projet

Ce travail souligne l'importance de ne pas se fier uniquement à l'exactitude (Accuracy) mais de privilégier des métriques comme l'AUC-PR et le rappel. Il offre une base solide pour le développement d'outils d'aide à la décision en gestion des risques.

---

11. Recommendations

11.1. Applications en banque

- **Système de "Shadowing" :** Déployer le modèle en parallèle du système actuel pour comparer les alertes générées avant une mise en production réelle.
- **Alertes en temps réel :** Intégrer le modèle dans le flux d'autorisation des transactions pour bloquer instantanément les cas à haut risque.

11.2. Améliorations futures

- **Apprentissage en ligne (Online Learning) :** Mettre en place un système de réentraînement automatique pour s'adapter aux nouvelles typologies de fraude.

- **Modèles hybrides :** Combiner la Forêt Aléatoire avec des modèles de détection d'anomalies non supervisés pour détecter des fraudes totalement inédites.

---

12. Annexes

12.1. Code Python (Résumé)

Le projet a été développé en Python en utilisant les bibliothèques standards de la Data Science :

- `Pandas` et `Numpy` pour la manipulation des données.
- `Scikit-learn` pour la modélisation et l'évaluation.
- `Imbalanced-learn` pour l'implémentation de SMOTE.
- `Matplotlib` et `Seaborn` pour les visualisations.

12.2. Détails techniques

Les modèles ont été entraînés sur un environnement virtualisé avec 80% des données pour l'apprentissage et 20% pour la validation finale. La reproductibilité a été assurée par la fixation d'une graine aléatoire (random_state=42).

Fin du Rapport Académique